

Practical:6

Implementation of chain matrix multiplication using dynamic programming.

/08/2026

Aim:

Implementation of chain matrix multiplication using dynamic programming.

Algorithm:

Algorithm

Input : number of items, their weights and profit, and the maximum capacity of knapsack

Output : Maximum profit that can be obtained without exceeding the knapsack capacity

Start

Step 1 : Read the number of items n

Step 2 : Read the weight $w[i]$ and profit $p[i]$ of each item.

Step 3 : Read the maximum capacity w of the knapsack.

Step 4 : Create a DP table $K[n+1][w+1]$

Step 5 : Initialize the first row and first column of the DP table with 0.

Step 6 : for each item $i=1$ to n , and for each capacity $j=1$ to w :

- If $w[i-1] \leq j$, calculate:
$$K[i][j] = \max(p[i-1] + K[i-1][j-w[i-1]], K[i-1][j])$$
- otherwise
$$K[i][j] = K[i-1][j]$$

Step 7 : Repeat the process until all items and capacities are considered

Step 8 : The maximum profit is stored in $K[n][w]$

Step 9 : Display the maximum profit

Step 10 : Stop

Program:

```
#include <iostream>
#include <climits>
using namespace std;

void printTable(int m[][100], int n) {
    cout << "\nDP Table (Minimum Costs):\n";

    for (int i = 1; i < n; i++) {
        for (int j = 1; j < n; j++) {
            if (i > j)
                cout << " - ";
            else
                cout << m[i][j] << "\t";
        }
        cout << endl;
    }
}

int matrixChainOrder(int p[], int n) {
    int m[100][100];

    // Cost of multiplying one matrix is 0
    for (int i = 1; i < n; i++)
        m[i][i] = 0;

    // L = chain length
    for (int L = 2; L < n; L++) {

        for (int i = 1; i < n - L + 1; i++) {

            int j = i + L - 1;

            m[i][j] = INT_MAX;
```



```
// Try every possible split
for (int k = i; k < j; k++) {

    int q = m[i][k]
        + m[k + 1][j]
        + p[i - 1] * p[k] * p[j];

    if (q < m[i][j])
        m[i][j] = q;
    }
}

printTable(m, n);

return m[1][n - 1];
}

int main() {

    int arr[] = {3, 5, 10, 7, 2};

    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Minimum number of multiplications is "
        << matrixChainOrder(arr, n) << endl;

    return 0;
}
```

Output:

```
for (int i = 1; i < n; i++)
    m[i][i] = 0;

Minimum number of multiplications is
DP Table (Minimum Costs):
0      150      360      270
- 0      350      240
- - 0 140
- - - 0
270

Process returned 0 (0x0)    execution time : 0.492 s
Press any key to continue.
```

Time complexity:

TIME Complexity

for (int length = 2; length <= n; length++) — $n-1$

for (int i = 1; i <= n - length + 1; i++) — $\frac{n(n-1)}{2}$

for (int k = 1; k < j; k++) — n

$$T(n) = n-1 + \frac{n(n-1)}{2} \times O(n)$$

$$= \frac{n^2 - n}{2} \times O(n)$$

ignoring constant

$$= n^2 \times O(n)$$

$$T(n) = O(n^2) \times O(n)$$

$$T(n) = O(n^3)$$

Worst case
All possible chain lengths, subchains, and split positions are evaluated
 $T(n) = O(n^3)$

Average case
The algorithm evaluates all DP subproblems and their possible split position
 $T(n) = O(n^3)$

Best case
Even if the first split happens to produce the optimal multiplication cost, the remaining split positions are still checked
 $T(n) = O(n^3)$

Conclusion:

The Matrix Chain Multiplication problem was successfully implemented using Dynamic Programming. The algorithm finds the most efficient order of multiplying matrices by minimizing the total number of scalar multiplications. It avoids repeated calculations and provides an optimal solution efficiently.